Foundations and Frontiers of Boosting

DS6410: Machine Learning II

Introduction to Ensemble Learning

CORE PHILOSOPHY

Combining multiple "weak" models to create a single "strong" model. The "wisdom of the crowd" applied to machine learning.

THE BIAS-VARIANCE TRADEOFF

- **Bagging**: Reduces **Variance** by averaging independent models (Parallel).
- **Boosting**: Reduces **Bias** by sequentially correcting errors (Sequential).

VISUALIZATION RECAP

- **Single Stump**: High bias, low variance.
- **Bagging**: Smooths out variance but limited by the base learner's bias.
- **Boosting**: Iteratively carves out complex decision boundaries.

> Key Insight: Boosting transforms a collection of simple rules into a highly accurate decision boundary.

The Boosting Framework

ADDITIVE MODELING

The final ensemble model is constructed as a weighted sum of M base learners:

$$F(x) = \sum_{m=1}^{M} \alpha_m h_m(x)$$

Where $h_m(x)$ are weak learners and α_m are their corresponding weights.

SEQUENTIAL OPTIMIZATION

1. Initialize the model with a constant value: $F_0(x)$.
2. Iterate for $m = 1$ to M.
3. Find a new weak learner $h_m(x)$ that minimizes the residual error of the previous model.
4. Update the ensemble: $F_m(x) = F_{m-1}(x) + \alpha_m h_m(x)$.

AdaBoost: The Classic Algorithm

1. Initialize observation weights: $w_i = 1/N$
2. For $m = 1$ to M:

→ Fit weak learner $h_m(x)$ using weights w_i

→ Compute weighted error rate:

$$\varepsilon_m = \sum w_i \, I(y_i \neq h_m(x_i)) \, / \sum w_i$$

→ Compute learner importance:

$$\alpha_m = ln((1 - \varepsilon_m) / \varepsilon_m)$$

→ Update weights:

$$w_i \leftarrow w_i \cdot exp(\alpha_m \cdot I(y_i \neq h_m(x_i)))$$

The Core Intuition

AdaBoost is **adaptive** because it modifies the data distribution at each step.

→ Observations that are **hard to classify** (misclassified in round m) receive higher weights.

→ The next learner h_{m+1} is forced to focus on these difficult cases.

→ Final prediction is a **weighted majority vote** of all weak learners.

AdaBoost: Intuition & Reweighting

REWEIGHTING MECHANISM

Multiplicative Updates

Weights are adjusted by a factor of $exp(\alpha_m)$ for misclassified points. As the error ε_m decreases, the penalty for mistakes grows exponentially.

Focusing Attention

The algorithm "zooms in" on the most difficult regions of the feature space, forcing subsequent learners to solve the hardest parts of the problem.

BOUNDARY EVOLUTION

From Stumps to Surfaces

A single decision stump provides a linear split. By combining many stumps with varying weights, AdaBoost carves out highly non-linear and complex decision surfaces.

Weighted Voting

The final model is not a simple average; it is a committee where the most accurate members (high α) have the loudest voice in the final decision.

Intuition: Boosting is like a student who studies only the questions they got wrong on the previous practice test.

Theoretical Foundations

Margin Theory

Boosting often avoids overfitting even when training error reaches zero. This is explained by the **Margin**: the confidence of the classification.

Boosting continues to increase the margin of training samples, which leads to better generalization bounds.

$$margin(x, y) = y \sum \alpha_m h_m(x)$$

Exponential Loss

AdaBoost can be derived as a **Forward Stage-wise Additive Model** that minimizes the exponential loss function:

$$L(y, f(x)) = exp(-y f(x))$$

This loss is a differentiable upper bound of the 0-1 loss, making it computationally efficient for gradient-based updates.

Boosting is a greedy optimization process that converges to the optimal solution under the "Weak Learnability" assumption.

Gradient Boosting Machines (GBM)

BOOSTING AS GRADIENT DESCENT

Gradient Boosting generalizes the sequential additive framework by treating the optimization as a gradient descent in **function space**. We minimize a differentiable loss function L(y, F(x)).

PSEUDO-RESIDUALS

Instead of re-weighting, we fit the next model to the negative gradient:

$$r_{im} = - [\partial L(y_i, F(x_i)) / \partial F(x_i)]_{F=F_{m-1}}$$

COMMON LOSS FUNCTIONS

- → **Regression:** Mean Squared Error (MSE) or Mean Absolute Error (MAE).
- → **Classification:** Log-Loss (Cross-Entropy) for probabilistic outputs.
- → **Robustness:** Huber loss for handling outliers in regression tasks.

KEY ADVANTAGE

This framework allows for the use of **any differentiable loss function**, making it extremely flexible for diverse machine learning tasks beyond simple classification.

GBM: Functional Gradient Descent Details

FITTING PSEUDO-RESIDUALS

At each step m, we compute the negative gradient (pseudo-residuals) for each observation:

$$r_{im} = - [\partial L(y_i, f(x_i)) / \partial f(x_i)]_{f=f_{m-1}}$$

- We train a base learner h_m to fit these residuals:
- $h_m = argmin \sum (r_{im} - h(x_i))^2$
- This "projects" the gradient into the hypothesis space of our weak learners.

FUNCTIONAL UPDATE

The model is updated by taking a step in the direction of the fitted weak learner:

$$f_m(x) = f_{m-1}(x) + \nu \cdot h_m(x)$$

- This is equivalent to performing gradient descent in the space of functions.
- The step size ν (shrinkage) is a crucial regularization parameter.
- Applicable to any differentiable loss function.

GBM: Shrinkage and Regularization

SHRINKAGE (STEP SIZE)

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

- The parameter **$\nu \in (0, 1]$** controls the learning rate.
- **Rule of Thumb:** Smaller step sizes ($\nu \approx 0.1$) generally lead to better generalization.
- Trade-off: Reducing ν requires a proportional increase in the number of iterations **M**.

STOCHASTIC BOOSTING

- **Bag Fraction:** At each stage, fit the base learner on a random subset of the training data (typically 50%).
- Reduces computation time and acts as a powerful form of **regularization**.
- Can be viewed as a **minibatch method** where sample size is independent of training set size.

Modern Frontiers: XGBoost & Regularization

REGULARIZED OBJECTIVE

$$Obj(\Theta) = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

$$\Omega(f) = \gamma T + \&frac\{1\}\{2\} \lambda \, ||w||^2$$

XGBoost (Chen & Guestrin, 2016) introduces a regularized objective to control model complexity, where T is the number of leaves and w are leaf weights.

ALGORITHMIC INNOVATIONS

- → **Second-Order Taylor Expansion:** Uses both gradient and Hessian for faster convergence and custom loss support.
- → **Sparsity-Aware Splitting:** Efficiently handles missing values and sparse data by learning default directions.
- → **System Optimizations:** Cache-aware data access, block compression, and parallel tree building.

LightGBM & CatBoost: Speed & Categorical Features

LightGBM (Microsoft)

GOSS

Gradient-based One-Side Sampling: Keeps instances with large gradients and performs random sampling on instances with small gradients to reduce data size while maintaining accuracy.

EFB

Exclusive Feature Bundling: Bundles mutually exclusive features (features that rarely take non-zero values simultaneously) to reduce the number of features.

Uses Leaf-wise tree growth for better accuracy on complex datasets.

CatBoost (Yandex)

Ordered Boosting

A permutation-based approach to train models on different subsets of data, effectively overcoming the prediction shift problem common in traditional GBDT.

Categorical Handling

Uses Ordered Target Statistics to transform categorical features into numerical values without target leakage, avoiding the curse of dimensionality from one-hot encoding.

Uses Symmetric Trees for extremely fast inference and model deployment.

Summary & Conclusion

01

Transformation

Boosting effectively transforms a collection of weak learners into a single, highly accurate strong predictive model.

02

Sequential Process

It is a sequential, additive process where each new model is specifically trained to correct the errors of its predecessors.

03

Practical Power

Modern implementations like XGBoost and LightGBM remain the gold standard for high-performance ML on tabular data.

Boosting is not just an algorithm, but a powerful framework for reducing bias and mastering complex decision boundaries.